

Assignment 3 – Network file system

Develop a (simple) network file system which allows applications to create and access files on a remote machine, in a transparent way. The desired functionality must be provided through primitives similar to those of the local file system:

<code>int mynfs_init(char *server_addr, int server_port, int blockSize, int maxCachedBlocks, int freshT);</code>	Initialize server contact info, the block size, the maximum number of file blocks to be kept in cache, and the block freshness time.
<code>int mynfs_open(char *fname);</code>	Open file for reading/writing. If the file does not exist, it is created.
<code>long mynfs_read(int fd, void *buf, long n);</code>	Read file.
<code>long mynfs_write(int fd, void *buf, long n);</code>	Write file.
<code>long mynfs_lseek(int fd, long pos, int whence);</code>	Move read/write position.
<code>int mynfs_truncate(int fd, long len);</code>	Truncate the file.
<code>int mynfs_close(int fd);</code>	Close file.

Client-side functionality. The above API must be supported via a suitable underlying request-reply protocol, including the following basic interactions: (1) send request to open/create a file and receive a file identifier and metadata (last modification timestamp and file size), (2) send request to read data from a given file identifier and position and receive a block of data and file metadata, (3) send request to write data in a given file identifier and position along with the data and receive the file metadata, (4) send request to truncate the file and receive the file metadata. These interactions must be implemented on top of UDP/IP with at-least-once semantics. The client side must run independently of the application threads.

Server-side functionality. The server does not provide an API. At startup, it takes as an argument the directory of the local file system that should be used as the root of the file system exported/accessible to the clients. The server maps each opened file to an internal identifier that is used as a reference in the above client-server interactions. The server should not maintain any client-specific information and should be able to handle client requests in a robust way even after a reboot (think how this can be done in a practical way without the server having to remember all the file identifiers issued in its previous lifetime).

Client-side cache. To accelerate read access, the client side receives from the server entire blocks of data independently of the number of bytes the application requested to read. These blocks are kept in the client's memory so that they can be used if the application asks to read more (consecutive) bytes in the future. The validity of such blocks must be checked in the spirit of NFS (on the server side, use the modification timestamp of the file system). The cache management parameters are specified via the `mynfs_init()` function. Use any cache eviction policy you wish. You should implement a write-through policy, i.e., the data that is written by the application is sent immediately to the server, which in turn performs the corresponding write in the respective file before sending the reply to the client.

Test / Measurements. Develop one or more simple applications that can be used to test your implementation and confirm correct operation in the presence of one or more clients accessing the same or different files concurrently. Record the file access speed and the number of client-server interactions and the amount of data that is sent over the network for different file read/write scenarios as a function of the block/cache size and freshness time, compared with the baseline case where the client does not have a cache.

Implementation details. The server side must be implemented in C using the standard system calls for the local file system. For the client side (and client test applications) you may use whichever programming language you wish. However, you may not make any assumption regarding the contents of the files that will be accessed by the application. Your test applications should take the files to be created/accessed as arguments (do not hardwire filenames in the application code).

Optional pet project. Extend your implementation so that the contents of a file can be transparently distributed across different servers in the form of non-overlapping file fragments. To this end, you must tackle several issues: how a client discovers/learns the fragmentation scheme of a file (which servers are responsible for which fragments); how this information is updated on the servers and the clients when the file size changes (new fragments are created or existing fragments are deleted/merged); the policy used to decide when to create/delete/merge fragments and how to distribute the file fragments on the available servers.

Submission deadline (see instructions in eclass): Saturday 2 May 2026, 22:00
